

Network Security Theory Revision Notes

Concepts, Attacks & Defenses — Complete Course Coverage

ICT2217 Network Security

Consolidated from all lecture notes and reference readings

Switch Security • ACLs & Firewalls • DoS/DDoS • AAA • Monitoring

Contents

Module 1 — Internal LAN Attacks & Switch Security

- 1.1 MAC Flooding Attack & Defense (Port Security)
- 1.2 Switch Spoofing — VLAN Hopping I (DTP)
- 1.3 Double-Tagging — VLAN Hopping II (Native VLAN)
- 1.4 STP Attack & Defense (BPDU Guard, Root Guard)
- 1.5 DHCP Attacks & Defense (DHCP Snooping)
- 1.6 ARP Poisoning, MitM & Defense (Dynamic ARP Inspection)
- 1.7 Address Spoofing & Defense (IP Source Guard)

Module 2 — Traffic Filtering, Firewalls & DoS/DDoS

- 2.1 Access Control Lists (Standard, Extended, Named, Time-based, Object-group)
- 2.2 Firewalls & the Cisco ASA/Firepower
- 2.3 Reconnaissance — Ping Sweeping & Port Scanning
- 2.4 Amplification & Reflection Attacks (Smurf)
- 2.5 TCP SYN Flooding (TCP Intercept & SYN Cookies)
- 2.6 Slowloris (Low-rate DoS)

Module 3 — Secure Management, AAA & Monitoring

- 3.1 Secure Remote Access — OOB, VRF, Jump Host, SSH
- 3.2 AAA Part 1 — Authentication
- 3.3 AAA Part 2 — Authorization
- 3.4 AAA Part 3 — Accounting
- 3.5 Security Monitoring — Syslog with NTP
- 3.6 Cisco Flexible NetFlow

Note: This document is the **theory companion** to the course. It explains every concept, how each attack works, and the defensive mechanism, with the most important configuration commands shown inline. A separate **Cisco Configuration Command Reference** collects the full command set for hands-on revision.

Module 1 | Internal LAN Attacks & Switch Security

Layer-2 is often the softest part of an enterprise network: switching protocols were designed for convenience, not security. This module covers the classic insider attacks that operate **within a LAN** and the switch features that defend against them. A recurring theme is that most L2 defenses build on one another — DHCP snooping underpins Dynamic ARP Inspection and IP Source Guard.

1.1 MAC Flooding Attack & Defense

Background — how a switch learns

A switch forwards frames using its **MAC address table** (CAM table). When the destination MAC of a frame is *not* found in the table, the switch **floods** the frame out of every port except the one it arrived on. Through **address self-learning**, the switch records source MACs so that later unicast frames are forwarded only to the intended port — which is why a switch is considered more ‘secure’ than a hub.

Crucially, the MAC address table is **finite in size**. Entries also **age out** automatically if not seen again, with a default aging time of **300 seconds**. Multicast addresses are never learned as source addresses (in the IEEE 802.3 header, bit 0 of the first octet = 1 marks a multicast MAC).

The attack

A **MAC flooding attack** bombards the switch with frames carrying **fake, random source MAC addresses** until the MAC address table is **full**. Once full, the switch can no longer learn the real destination, so authentic unicast frames are **flooded to all ports** — including the attacker’s — effectively turning the switch into a hub and allowing traffic to be sniffed. The tool **macof** (from the dsniff suite, available in Kali Linux) generates such frames. To succeed, the attacker must flood enough frames **within the aging time** before entries expire.

Defense — Port Security

Port Security restricts a switch interface by limiting which, and how many, MAC addresses are allowed on it. It is configured on **access** ports with a combination of these options:

- **Maximum** — the maximum number of MACs allowed on the port.
- **Static** — a specific MAC address is manually configured and permitted.
- **Sticky** — dynamically learned MACs are converted to (‘stuck’ into) the running configuration automatically.
- **Dynamic** — learned MACs are kept only until the switch reboots (default behaviour).

Enabling port security (key commands):

```
Switch(config)# interface interface-id
Switch(config-if)# switchport mode access
Switch(config-if)# switchport port-security
Switch(config-if)# switchport port-security maximum value
Switch(config-if)# switchport port-security mac-address sticky
```

Violation modes

Three actions can be taken when a violation (an unexpected MAC, or exceeding the maximum) is detected:

- **Protect** — silently drops offending frames; no notification.
- **Restrict** — drops offending frames and generates a log/SNMP notification and increments a violation counter.
- **Shutdown** (default) — puts the port into the **error-disabled (err-disabled)** state, shutting it down entirely.

Verification uses `show port-security` and `show port-security interface interface-id`.

Recovering err-disabled ports

- **Manual recovery** (more control): shutdown then no shutdown on the interface.
- **Auto recovery** (less admin overhead): configure `errdisable recovery cause psecure-violation` and an `errdisable recovery interval`.

Note: Port Security is the answer to MAC flooding, but note (Lab 1.5) it does **not** stop DHCP starvation, because an attacker can vary the DHCP payload MAC while keeping one source MAC on the frame.

1.2 Switch Spoofing — VLAN Hopping I

Background — access vs. trunk and DTP

A switch port operates as either an **access** port (carries frames for a single VLAN) or a **trunk** port (carries multiple VLANs over one physical link, using 802.1Q tagging). To simplify setup, Cisco switches run the **Dynamic Trunking Protocol (DTP)**, which can negotiate a link into a trunk automatically. A Cisco port can be in one of four modes: **access**, **trunk**, **dynamic auto**, or **dynamic desirable**. By default many ports are in **dynamic auto**, which passively waits to form a trunk if the other side asks.

The attack

Switch spoofing is a form of **VLAN hopping**. Exploiting the default **dynamic auto** setting, an attacker injects **DTP messages** to negotiate the port into a **trunk**. With a spoofed trunk, the attacker can now **receive frames from all VLANs** and **send frames tagged for any target VLAN** — crossing VLAN boundaries that should have isolated them. The tool **Yersinia** (Kali Linux) can perform this.

Defense

- **Manually configure** ports as access or trunk explicitly (do not leave them dynamic).
- **Disable DTP** negotiation on the port with `switchport nonegotiate`.
- Good practice: **shut down all unused ports** and place them in an **unused VLAN** (e.g. VLAN 100) so a live, exploitable port cannot be discovered.

Core defense commands:

<code>Switch(config-if)# switchport mode access</code>	<code>! or 'trunk' – set explicitly</code>
<code>Switch(config-if)# switchport nonegotiate</code>	<code>! disable DTP</code>

1.3 Double-Tagging — VLAN Hopping II

Background — native VLAN and 802.1Q

Inside a trunk, IEEE **802.1Q** distinguishes VLAN traffic by tagging. 802.1Q defines two kinds of VLAN on a trunk: one **native VLAN** whose frames are sent **untagged**, and all other VLANs whose frames are **tagged**. The 802.1Q tag also carries a 3-bit priority field used for Class of Service (CoS) under **802.1p**, which supports Voice over IP — hence access ports for phones are configured with a **data VLAN** and an auxiliary **voice VLAN**.

The attack

Double-tagging is another VLAN-hopping attack that lets an attacker send frames into another VLAN **without going through a router**. The attacker crafts a frame with **two** 802.1Q tags: an outer tag matching the trunk's native VLAN and an inner tag for the target VLAN. The first switch strips the (native-matching) outer tag and forwards the still-tagged frame over the trunk; the next switch then delivers it to the target VLAN. It is unidirectional and works **only if the attacker's access port is in the same VLAN as the trunk's native VLAN** (default native VLAN is 1).

Defense

- Configure a **dedicated native VLAN ID** for trunks (e.g. VLAN 500) that is **never used for any access port**:
`switchport trunk native vlan 500`.
- **Enable tagging of the native VLAN** where supported (forces all trunk traffic, including native, to be tagged): `vlan dot1q tag native`. Newer Catalyst 9200L/9300L switches support forcing tagging of all trunk traffic.
- Good practice: **do not use the default VLAN 1**.

1.4 STP Attack & Defense

Background

For availability, switches are interconnected with **redundant links**, and IEEE 802.1w **Rapid Spanning Tree Protocol (RSTP)** runs automatically to prevent loops by blocking redundant paths. On a link failure, RSTP re-converges so a previously blocked link becomes forwarding, keeping the network up.

The attack

RSTP has **no security**. An attacker connects a device (e.g. a Kali PC bridging two switch ports) and injects **RSTP BPDUs** (Bridge Protocol Data Units) advertising a **lower bridge priority** to win the root-bridge election. Once the attacker becomes the root bridge, traffic is re-routed through the attacker's machine and can be sniffed.

Defense

- **BPDUGuard** — on any port that should never receive BPDUs (i.e. ports facing user PCs), receiving a BPDU puts the port into **err-disabled** state. Per-interface: `spanning-tree bpduguard enable`; or globally on all PortFast ports: `spanning-tree portfast bpduguard default`.
- **PortFast** — lets an access port skip the RSTP listening/learning delay and go straight to forwarding, since a host cannot create a loop: `spanning-tree portfast`.
- **Root Guard** — on ports that should always be designated (typically distribution-switch ports toward access switches). If a **superior BPDU** arrives, the port transitions to **root-inconsistent** and blocks until the superior BPDU stops, preserving the intended root placement: `spanning-tree guard root`.

Err-disabled ports are recovered manually (`shutdown / no shutdown`) or automatically via `errdisable recovery cause ...` and `errdisable recovery interval`.

Note: BPDUGuard vs. Root Guard: a BPDUGuard port err-disables on *any* BPDU; a Root-Guard port still accepts BPDUs and only blocks when a *superior* BPDU threatens to take over as root.

1.5 DHCP Attacks & Defense

Background

DHCP lets hosts automatically obtain an IP address, default gateway, DNS server and more. DHCP has **no built-in security**, which enables two chained attacks.

Attack 1 — DHCP starvation

The attacker floods the server with many **DHCPDISCOVER** messages to **exhaust the entire pool of IP addresses** (a denial of service). The tool **DHCPig** is used in the lab. Port Security is **not** an effective defense: an attacker keeps a single frame source MAC (satisfying Port Security) while varying the **client hardware address (chaddr)** inside the DHCP payload, because DHCP operates at the application layer above the frame header.

Attack 2 — DHCP server spoofing (→ DNS & website spoofing)

After starving the legitimate server, the attacker runs a **rogue DHCP server** (e.g. the Metasploit DHCP module) to answer clients. By handing out a **rogue DNS server** address, the attacker can mistranslate domain names and redirect victims to a **spoofed website** to harvest credentials. A rogue DNS service such as **DNSChef** completes the chain.

Defense — DHCP Snooping

DHCP snooping lets a switch perform deep inspection of DHCP messages. When enabled, **all ports are untrusted by default** and will not forward server-side messages (DHCP OFFER/DHCP ACK), making a rogue server impossible. Only ports toward the **authorized DHCP server** are set **trusted**.

Key DHCP snooping commands:

```
Switch(config)# ip dhcp snooping
Switch(config)# ip dhcp snooping vlan vlan-id
Switch(config)# ip dhcp snooping verify mac-address      ! chaddr must match frame source MAC
Switch(config-if)# ip dhcp snooping trust                ! only on the authorized-server port
Switch(config-if)# ip dhcp snooping limit rate rate      ! 1-2048 DHCP pkts/s; err-disable if exceeded
```

- **Rate-limiting** (1–2048 pkts/s) per port; exceeding it err-disables the port.
- **MAC-address verification** checks that the frame source MAC matches the DHCP payload chaddr and drops if they differ.
- A switch with snooping inserts an **option-82** (relay agent information) field — circuit ID (VLAN + port) and remote ID (base MAC) — giving client-location info. Because the switch is not a relay agent, the giaddr stays zero; a downstream relay/router may drop such packets by default, so either trust them (ip dhcp relay information trust-all) or disable option-82 (no ip dhcp snooping information option).
- Snooping builds a **DHCP snooping binding table** (MAC ↔ IP ↔ VLAN ↔ port), added on DHCP ACK and removed on DHCP NAK/RELEASE/DECLINE. This table is the foundation for DAI and IP Source Guard.

1.6 ARP Poisoning, MitM & Defense

Background

ARP resolves an IP address to a MAC address. A **gratuitous ARP (GARP)** is an unsolicited ARP reply sent without a request — legitimately used, for example, by an active HSRP router to announce ownership of the virtual MAC.

The attack

ARP poisoning misuses gratuitous ARP to plant false IP→MAC mappings in victims' ARP caches. By poisoning **both** a victim host and the default gateway simultaneously, the attacker sits in the middle of all their traffic — a **Man-in-the-Middle (MitM)** attack. All traffic then flows through the attacker and can be captured, including plaintext usernames and passwords. The tool **Bettercap** (Kali Linux) performs ARP poisoning and sniffing.

Defense — Dynamic ARP Inspection (DAI)

A switch already running **DHCP snooping** can add **DAI** to drop invalid ARP traffic on **untrusted** ports. After enabling DAI, all ports are untrusted by default; the MAC/IP in each received ARP packet is validated against the **DHCP snooping binding table** and dropped if invalid.

Key DAI commands:

```
Switch(config)# ip arp inspection vlan vlan-id
Switch(config)# ip arp inspection validate {src-mac | dst-mac | ip} ! optional deeper checks
Switch(config-if)# ip arp inspection trust                          ! trusted uplink ports
Switch(config-if)# ip arp inspection limit rate rate                ! default 15 pps; err-disable if exceeded
```

- Each port can be given an **ARP rate-limit** (default 15 pps); exceeding it err-disables the port.
- For a **non-DHCP environment** (no binding-table entry), DAI is driven by an **ARP ACL** that manually maps IP↔MAC, applied with `ip arp inspection filter arp-acl vlan vlan-id [static]`. The optional **static** keyword drops packets that do not match the ACL instead of falling back to the binding table.

1.7 Address Spoofing & Defense

The attacks

Address spoofing may use **IP** spoofing, **MAC** spoofing, or both.

- **IP address spoofing** — an insider changes their PC's IP to that of a permitted host (e.g. a manager 'Bob') to **bypass an ACL** and reach a restricted server. IP spoofing also underlies TCP SYN flooding and Smurf attacks.
- **MAC address spoofing** — feasible only within a LAN. An insider clones a server's MAC and sends a frame to update the access switch's MAC table so that frames destined for the server are delivered to the attacker instead. This is one-way; the attacker may also spoof the server's IP and clone it to capture credentials.

Defense — IP Source Guard (IPSG)

IP Source Guard is configured on switch interfaces and permits traffic only from valid source addresses, using a **source binding table** that is populated automatically by DHCP snooping (or statically). It can protect against IP spoofing alone, or IP **and** MAC spoofing together.

Key IPSG commands:

```
Switch(config-if)# ip verify source          ! IP spoofing protection
Switch(config-if)# ip verify source mac-check ! IP + MAC spoofing protection
Switch# show ip verify source                ! verification
```

Note: For IPSG to check MAC as well as IP, **Port Security must also be enabled** on the interface. If DHCP snooping is not in use, add static bindings with `ip source binding mac vlan vlan-id ip interface int-id`.

Module 2 | Traffic Filtering, Firewalls & DoS/DDoS

Having secured Layer 2, this module moves to filtering IP traffic at Layer 3–4. It covers Access Control Lists, stateful firewalls (the Cisco ASA/Firepower), and the reconnaissance and denial-of-service attacks that filtering — and sometimes only a stateful firewall — can defend against.

2.1 Access Control Lists (ACLs)

What an ACL is

An **ACL** is a sequential list of **Access Control Entries (ACEs)**. Each ACE has an **action (permit or deny)** and a **test condition**. ACEs are processed **top to bottom**: on the first match the action is taken and processing stops; if nothing matches there is an **implicit 'deny any'** at the end that drops the packet. Anything not explicitly permitted is implicitly denied, so **ACE order matters** — place **more specific** entries above more general ones. An ACL has no effect until it is **applied to an interface**.

General rules

- **One ACL per direction** — filter inbound and outbound separately.
- **One ACL per interface** per direction (one 'in' and one 'out').
- **One ACL per protocol** (e.g. IPv4 vs. IPv6).
- **Inbound (ingress)** ACLs filter before the routing decision (saving CPU); **outbound (egress)** ACLs filter after routing, before the packet leaves. Packets the router originates are not filtered by its own outbound ACLs.

The five ACL types in this course

- **Standard numbered** (1–99, 1300–1999) — filters on **source IP only**; place near the **destination** to avoid over-filtering.
- **Extended numbered** (100–199, 2000–2699) — filters on protocol, source/destination IP and source/destination port; place near the **source** to save bandwidth.
- **Named** — identical function but uses a meaningful name instead of a number.
- **Time-based** — an ACL tied to a **time-range** so it is active only during set hours/days (references the router clock, which should be NTP-synchronised).
- **Object-group based** — groups hosts/subnets (**network** object groups) and protocols/ports (**service** object groups) for far more readable, manageable ACLs.

Wildcard (inverse) masks

Standard/extended ACLs specify addresses with a **wildcard mask** — the bit-inverse of the subnet mask (a 0 bit means 'must match', a 1 bit means 'don't care'). Shortcut: subtract the subnet mask from 255.255.255.255 (e.g. /24 → 0.0.0.255). Special keywords: **any** matches all addresses; **host** matches a single address. Newer IOS also accepts CIDR /prefix notation on standard ACLs.

Representative ACL syntax (see the Command Reference for the full set):

```

! Standard numbered
R(config)# access-list 10 permit 192.168.1.0 0.0.0.255
R(config-if)# ip access-group 10 in

! Extended (protocol / ports)
R(config)# access-list 100 deny tcp 10.3.0.0 0.0.255.255 host 10.1.0.1 eq 23
R(config)# access-list 100 permit ip any any

! Named + inline editing by sequence number
R(config)# ip access-list extended BLOCK_SALES
R(config-ext-nacl)# deny ip 192.168.50.0 0.0.0.255 any
R(config-ext-nacl)# permit ip any any

```

Extended ACLs use operators before port numbers: **eq** (equal), **gt** (greater-than), **lt** (less-than), **neq** (not-equal), and **range**. Ports may be given as numbers or well-known names (e.g. eq 80 or eq www). For ICMP, message-type keywords (e.g. **echo**) are used instead of eq. Use remark to comment an ACL and log to log matches; verify with show access-lists and show ip interface.

2.2 Firewalls & the Cisco ASA/Firepower

Firewall concepts

A firewall **segments a network into security zones** — typically an internal (trusted) zone, an external (untrusted) zone, and often a **DMZ** for servers reachable from the Internet. Three common types exist:

- **Packet-filtering firewall** — filters on L2/L3/L4 addresses and ports; a router with an ACL is a packet-filtering (stateless) firewall.
- **Stateful firewall** — maintains a **state/connection table** so return traffic is automatically allowed; in industry, 'firewall' usually means this type.
- **Application proxy firewall** — filters a specific application, e.g. a Web Application Firewall (WAF).

Cisco ASA / Firepower

Cisco's current **Firepower** next-generation firewalls (e.g. Firepower 1010) replaced the older **ASA**. NGFWs add IPS, advanced malware protection and threat intelligence (Cisco Talos). For study, the lab Firepower runs the **ASA** image to behave as a traditional stateful firewall. A firewall is usually deployed in **routed mode**. On the 1010, Ethernet 1/1 is a routed port (toward the edge router) and 1/2–8 are switch ports; a switch port is made routed with no switchport.

Security levels & the default policy

Each interface needs an IP, a **nameif** name and a **security-level** (0–100): 0 = least trusted (Internet), 100 = most trusted (inside), ~50 for a DMZ. **Default policy:** traffic initiated from a **higher** to a **lower** security level is allowed (and its return traffic permitted via the state table); traffic from **lower to higher** is denied. Two interfaces at the **same** security level cannot pass traffic unless same-security-traffic permit inter-interface is set.

Interface / security-zone configuration:

```

ciscoasa(config)# interface interface-id
ciscoasa(config-if)# nameif interface-name
ciscoasa(config-if)# security-level 0-100
ciscoasa(config-if)# ip address ip-addr subnet-mask

```

ACLs and Modular Policy Framework (MPF)

To let outside users reach a DMZ server (overriding the default deny), an **ACL** is applied on the outside interface — note ASA ACLs use a **subnet mask**, not a wildcard mask. Additional security features are configured with the **Modular Policy Framework (MPF)**, three steps that parallel ACL logic:

- **class-map** — identifies the traffic to match (like an ACL test condition).
- **policy-map** — defines the action on matched traffic (e.g. **inspect** an application engine, or **set connection** limits).
- **service-policy** — applies the policy-map, either **global** (one global policy) or per interface (which takes precedence).

Example: an ASA **drops ping by default**; enabling ICMP inspection in the default global policy (policy-map global_policy → class inspection_default → inspect icmp) lets ping replies return. **Packet order** in an ASA: state-table check → (if new) ACL check → MPF inspection → route to egress.

2.3 Reconnaissance — Ping Sweeping & Port Scanning

Reconnaissance is the first phase of the cyber kill chain. **Ping sweeping** (host discovery) narrows a large IP block to live hosts; **port scanning** (service discovery) then finds open ports/services to exploit. **Nmap** is the standard tool, and its options map neatly onto the protocol stack:

- **L2 — ARP scan** (default on the local subnet): sends ARP requests; cannot be filtered, so it reliably finds local hosts.
- **L3 — IP protocol scan** (-PO) and traditional **ICMP ping** (-PE, often blocked by firewalls).
- **L4 — TCP SYN scan** (-PS), **TCP ACK scan** (-PA, emulates return traffic to slip past packet filters), and **UDP scan** (-PU, solicits ICMP port-unreachable to infer a live host).

Nmap classifies ports as open / closed / filtered / unfiltered / open|filtered / closed|filtered based on responses (or their absence). A poorly configured firewall may leak information by returning ICMP 'communication administratively prohibited'.

Defense

- A naïve ACL blocking all inbound scans also blocks **return traffic** for internal users. Instead use the **established** keyword to permit only return packets (ACK/RST/SYN-ACK/FIN set): `permit tcp any 200.1.1.0 0.0.0.255 established`. ACLs can also match specific TCP flags (match-all +ack, etc.).
- Stop information leakage on drop with `no ip unreachable` on the interface.
- Best defense: a **stateful firewall**, which by default permits only return traffic and denies all Internet-initiated traffic (except where an ACL explicitly permits access to DMZ servers).

2.4 Amplification & Reflection Attacks (Smurf)

DoS/DDoS falls into two broad categories: **volumetric** floods (overwhelm with traffic, often via a **botnet**) and **low-rate** attacks. Even without a botnet, an attacker can generate a volumetric flood using **amplification and reflection**: spoof the **target's source IP** and send small requests to servers that reply with much larger responses, reflecting amplified traffic onto the target. UDP services such as **NTP** and **DNS** are especially abusible.

The classic example is the **Smurf attack**: a single **broadcast ping** is sent to a directed broadcast address with the target's spoofed source IP, so every host in that subnet reflects a ping reply to the target. The lab uses **hping3** to demonstrate it (after intentionally re-enabling directed broadcast, which is disabled by default today).

Defense

- Modern defense: **disable directed broadcast** on routers and disable broadcast-ping responses in the OS (this is why Smurf is largely mitigated today).

- The key enabler is **source IP spoofing**, so adopt **RFC 2827 / BCP 38**: ISPs filter packets whose source IP is outside a customer's allocated block. Enterprises should likewise filter at edge routers (and distribution switches) any packets whose source IP does not match their allocated block, plus **anti-spoofing ACLs** that deny known-bogus source addresses.

2.5 TCP SYN Flooding

TCP establishes connections via the **3-way handshake** (SYN → SYN/ACK → ACK). A **SYN flooding attack** sends many SYNs with **spoofed source IPs**; the server's SYN/ACKs are never answered, leaving many **half-open (embryonic)** connections in the **SYN-RECV** state that consume server resources until legitimate users can no longer be served. Each half-open connection ties up a **Transmission Control Block (TCB)** in memory.

Defense — TCP Intercept & SYN cookies (ASA/Firepower)

ACLs cannot stop SYN flooding. Cisco ASA/Firepower firewalls implement **TCP Intercept**, activated when an **embryonic-connection limit** is crossed (configured via MPF set `connection embryonic-conn-max` and `per-client-embryonic-max`). TCP Intercept acts as a **proxy**: it answers the client's SYN itself; only if a valid ACK returns does it complete the real connection to the server. This shifts the flood from the server to the firewall.

To protect the firewall itself, the **SYN cookie** technique encodes the TCB into the 32-bit initial sequence number of the SYN/ACK (using a cryptographic hash such as SHA-512 keyed by a pool of local secrets), so **no memory is consumed** for unanswered SYNs. If an ACK returns, the cookie is validated (the secret makes it a message authentication code that an attacker cannot forge) and the TCB is reconstructed. Caveat: TCP Intercept and cookie computation are **CPU-intensive**, so a sufficiently large flood can still overwhelm the firewall. Enable monitoring with `threat-detection statistics tcp-intercept`.

2.6 Slowloris (Low-rate DoS)

Unlike volumetric floods, **Slowloris** is a **low-rate** attack needing only a few crafted packets. It exploits a design feature of **HTTP**: the server waits for the **complete request header** before responding, holding the connection open meanwhile. The attacker opens many concurrent connections and sends **incomplete headers very slowly**, keeping every connection reserved until the server's connection pool is exhausted and legitimate users are locked out. A single PC (using the **slowhttptest** tool) can bring down a web server.

Defense

ACLs cannot help (the ACL is deliberately permitting web users, who include the attacker). The defense is again the stateful firewall's ability to **limit connections** via MPF (set `connection` parameters — e.g. `per-client` and `embryonic/idle` limits — tuned experimentally), applied with a class-map and service-policy on the required interface.

Module 3 | Secure Management, AAA & Monitoring

The final module secures how devices are **managed** and how activity is **controlled and observed**: out-of-band management with VRFs and a jump host, SSH, the three A's of AAA (Authentication, Authorization, Accounting), and centralised monitoring with Syslog, NTP and NetFlow.

3.1 Secure Remote Access — OOB, VRF, Jump Host, SSH

Out-of-Band (OOB) management

Managing devices remotely over the production LAN (in-band) exposes management traffic to attack.

Out-of-band (OOB) management uses a **separate management LAN**, isolated from the production network. Many devices ship with a dedicated **management port**; where a device (e.g. the Catalyst 8200L router) has none, isolation is achieved with a **management VRF**.

Virtual Routing and Forwarding (VRF)

VRF gives a router multiple independent routing tables — logically splitting one router into several. Traffic in one VRF cannot cross into another, so a management VRF keeps management traffic separate from production traffic. VRFs also solve **overlapping customer address spaces** at service providers; VRF used without MPLS is called **VRF-Lite**. Once a VRF is created, verification/testing commands become **VRF-aware** — ping, traceroute, show ip route etc. must include the vrf vrf-name keyword.

Creating a VRF and assigning an interface:

```
! Newer syntax (Catalyst 8200 / IOS XE)
R(config)# vrf definition vrf-name
R(config-vrf)# address-family ipv4
R(config-if)# vrf forwarding vrf-name           ! then re-add the IP address

! Older IOS syntax
R(config)# ip vrf vrf-name
R(config-if)# ip vrf forwarding vrf-name
R# show ip route vrf vrf-name
```

Note: Assigning an interface to a VRF **deletes any IPv4 address already on it**, so the address must be reconfigured afterwards.

Jump host and SSH

A **jump host / jump server / jump box** is a hardened intermediary that is the single entry point from a less-secure zone (e.g. the Internet) into a more-secure zone (the OOB LAN), reducing the attack surface and making activity easy to log and audit. A typical design is a hardened Linux SSH host in the DMZ that is permitted to reach the OOB LAN. Its drawbacks are that it becomes an attractive target and a single point of failure — so it must be securely hardened.

Devices are managed with **SSH** (Telnet is insecure and avoided). Enabling SSH requires a hostname and domain name (for key naming), an RSA key pair (2048/4096-bit), and SSH version 2:

Enabling SSH for remote management:

```
R(config)# hostname host-name
R(config)# ip domain-name domain-name
R(config)# crypto key generate rsa modulus 2048
R(config)# ip ssh version 2
R(config-line)# login local
R(config-line)# transport input ssh
```

3.2 AAA Part 1 — Authentication

AAA = Authentication (who are you?), **Authorization** (what may you do?) and **Accounting** (what did you do?).

Local authentication

- **Simple password** on a line (everyone shares it): password + login on the console/VTY lines; enable password for privileged mode.
- **Local username/password** (required for SSH): username ... plus login local on the line.
- By default passwords are stored in **clear text**. service password-encryption gives only weak protection. Replacing password with **secret** stores an **MD5** hash; **sha256/scrypt** are stronger and recommended.
- Slow brute-force with login block-for ... attempts ... within ... (a 'quiet-mode' lockout), optionally excepting admins via login quiet-mode access-class; log attempts with login on-success/on-failure log.

Server-based AAA — TACACS+ and RADIUS

Storing credentials on a **centralised AAA server** scales far better than repeating them on every device and provides **fault tolerance** (multiple servers). Devices talk to the server using **TACACS+** or **RADIUS**. Enable AAA, define the server, then build a **method list** (the default list applies to VTY automatically; custom lists must be applied to a line).

Server-based authentication (TACACS+ example):

```
SW(config)# aaa new-model
SW(config)# tacacs server TACACSVR
SW(config-server-tacacs)# address ipv4 192.168.1.x
SW(config-server-tacacs)# key shared-secret
SW(config)# aaa authentication login default group tacacs+ local
```

Common method keywords include **local**, **local-case**, **enable**, **none**, **group radius**, **group tacacs+**. Listing several (e.g. group tacacs+ group radius local) gives fallback order so you are not locked out if servers are down.

TACACS+ vs. RADIUS

Functionality	TACACS+ separates authentication, authorization and accounting (modular). RADIUS combines authentication and authorization (less flexible).
Standard	TACACS+ is mostly Cisco. RADIUS is an open/RFC standard.
Transport	TACACS+ = TCP port 49 . RADIUS = UDP (auth 1812, acct 1813).
Security	TACACS+ encrypts the entire packet . RADIUS encrypts only the password .
Authorization	TACACS+ can authorize commands per-user/per-group. RADIUS has no such command option.
Primary use	TACACS+ → device administration. RADIUS → network access.

Because TACACS+ separates the functions, authentication happens in its own exchange; in RADIUS, authentication and authorization are carried together in the Access-Accept reply.

3.3 AAA Part 2 — Authorization

After authentication, users should get **no more rights than necessary**. Two models are covered.

Privilege levels

By default Cisco uses three levels: **0** (five basic commands), **1** (user EXEC, read-only commands like show) and **15** (privileged EXEC). Commands can be moved to custom levels **2–14**, and a password set per level. Users may be assigned a level so they enter directly at it. A user must be at the same or higher level than a command to run it.

Assigning commands / users to privilege levels:

```
R(config)# privilege exec level 5 ping
R(config)# enable secret level 5 cisco5
R(config)# username SUPPORT privilege 5 secret ...
! Via AAA server: authorize with
R(config)# aaa authorization exec default group tacacs+ group radius local
```

Role-Based CLI Views

Privilege levels have a flaw: lower-level commands are always available to higher-privilege users. **RBAC using views** lets you pick exactly which commands a role can run. Three view types: **Root view** (to create views, like level 15), **CLI view** (a defined command set), and optional **Superview** (groups CLI views but holds no commands itself). AAA must be enabled (`aaa new-model`); enter root view with `enable view`. A maximum of 15 views can be created.

Defining a CLI view:

```
R# enable view ! enter root view (enable secret password)
R(config)# parser view VIEWNAME
R(config-view)# secret view-password
R(config-view)# commands exec include show version
R(config-view)# commands exec include ping
```

Views can be assigned to users locally (`username X view ADMIN secret ...`) or centrally in the AAA server. As with authentication, TACACS+ authorizes separately from authentication, while RADIUS carries authorization together with authentication.

3.4 AAA Part 3 — Accounting

Accounting records what users did — crucial for security auditing. A useful type is **exec**, which logs EXEC terminal sessions: with **start-stop**, a start message is sent when the session begins and a stop message when it ends.

Configuring exec and command accounting:

```
R(config)# aaa accounting exec default start-stop group tacacs+ group radius
R(config)# aaa accounting commands 15 default start-stop group tacacs+
```

- For **RADIUS**, accounting is done separately after authentication/authorization; TACACS+ likewise accounts separately.
- **Command accounting** (which specific privilege-level commands a user ran) is supported by **TACACS+ only**, not RADIUS. **stop-only** sends a stop message when a command executes.

3.5 Security Monitoring — Syslog with NTP

Why NTP first

To correlate logs across devices, their clocks must agree — achieved with **NTP (Network Time Protocol)**, which runs over **UDP port 123**. In client/server mode the client computes clock offset and round-trip delay from timestamps in the reply. NTP is organised into a **stratum** hierarchy describing hops from an authoritative time source; the higher the stratum number, the lower the accuracy. UTC (=GMT) is the worldwide standard; public

time comes from sources such as the NTP Pool Project.

Devices can act as NTP **server**, **client**, **peer** (symmetric) or **broadcast**. When a device syncs to a source it automatically becomes a server for others (a stratum-1 sync makes it stratum-2).

Core NTP configuration & verification:

```
R(config)# ntp master [stratum]
R(config)# ntp server {ip | hostname}
R(config)# ntp peer {ip | hostname}
R# show ntp status ; show ntp associations
```

Because an attacker can inject false NTP replies to corrupt time (and thus log timings), enable **NTP authentication** (a shared key produces an MD5 hash the client verifies): `ntp authentication-key ... md5 ..., ntp trusted-key ..., ntp authenticate`. NTP can also be protected with ACLs via `ntp access-group`, and events logged with `ntp logging`. Over an OOB/VRF network, sync with `ntp server vrf Mgmt-vrf ....`

Syslog

Syslog (RFC 5424) is the standard for event-notification messages, sent by default over **UDP port 514** (or more securely over TLS/TCP 6514). Messages carry a timestamp, a **severity level** and a description. Severity ranges 0–7 (0 = emergencies, most severe; 7 = debugging, least severe); configuring a level logs that level **and all lower-numbered (more severe) levels**.

Sending logs to RAM and a central server:

```
R(config)# service timestamps log datetime msec
R(config)# logging buffered          ! log to RAM (show with 'show logging')
R(config)# logging host 172.16.3.9   ! log to a central Syslog server
R(config)# logging trap warnings     ! or a level 0-7
```

Housekeeping while configuring: `no logging console` stops console spam; `logging synchronous` (line) stops messages interrupting command entry; `terminal monitor` shows logs over a Telnet/SSH session. Centralising all alerts to a Syslog server or a **SIEM** enables analysis and correlation.

3.6 Cisco Flexible NetFlow

NetFlow collects statistics on IP packets flowing through a device, supporting security monitoring, traffic analysis, planning and IP accounting. A **flow** is a **unidirectional** sequence of packets sharing key header fields (e.g. source/destination IP, ports, protocol). Statistics are kept in a **NetFlow cache** and exported to a **NetFlow Collector** for centralised analysis. NetFlow has evolved into variants including Flexible NetFlow, IPFIX, sFlow and JFlow.

Flexible NetFlow (NetFlow v9) lets you customise what is matched and collected, via reusable components configured in four steps:

1. **Flow record** — defines what to collect: match fields (the flow keys) and collect counters (e.g. bytes, packets). Predefined records also exist (e.g. `ipv4 original-input`).
2. **Flow exporter** — defines **where** statistics are sent (destination collector IP, transport UDP port, export-protocol `netflow-v9`, source interface).
3. **Flow monitor** — associates a flow record with an exporter (and a cache timeout).
4. **Apply to interface** — `ip flow monitor NAME {input | output}` on the chosen interface.

Flexible NetFlow skeleton (full detail in the Command Reference):

```
R(config)# flow record my-record
R(config-flow-record)# match ipv4 source address
R(config-flow-record)# collect counter bytes
R(config)# flow exporter my-exporter
R(config-flow-exporter)# destination 10.1.10.100
R(config)# flow monitor my-monitor
R(config-flow-monitor)# record my-record
R(config-flow-monitor)# exporter my-exporter
R(config-if)# ip flow monitor my-monitor input
```

A **SIEM (Security Information and Event Management)** increasingly ties Syslog, NetFlow and other sources together for unified, real-time correlation of logs and threat alerts — the capstone of network security monitoring.